

Medical Image Classification Module

MedAssistAI

Kaushika Yadav G

1. Project Description

Medical Image Classification Module

The objective of this module is to develop an AI-based medical image classification system capable of identifying diseases from diagnostic medical images. The module forms a component of the MedAssistAI project and provides automated disease prediction using deep learning.

A reusable training pipeline was developed using TensorFlow and MobileNetV2 transfer learning. The pipeline supports multiple dataset formats and enables training, evaluation, model saving, and prediction using a common workflow.

The completed module currently supports four diseases:

- Glaucoma
- Brain Stroke
- Diabetic Retinopathy
- Heart Attack

The trained models are intended for integration into the backend inference pipeline.

2. Objective

The objectives of this work were:

- Develop a reusable image classification pipeline.
- Support both CSV-based and folder-based datasets.
- Train disease-specific deep learning models.
- Evaluate model performance using standard classification metrics.
- Save trained models for deployment.
- Provide a reusable prediction module for integration with the MedAssistAI system.

3. Dataset Used

The following publicly available datasets were utilized.

Disease	Dataset Format
Glaucoma	CSV
Brain Stroke	CSV
Diabetic Retinopathy	CSV
Heart Attack	Folder

During dataset analysis, additional datasets such as Pneumonia, Tuberculosis, Acne, Psoriasis, Skin Disease, Bone Fracture, Cataract, Chickenpox, and others were examined. Several datasets were excluded because of:

- Single-class labels
- Missing label columns
- Missing CSV annotations
- Severe class imbalance

Only datasets suitable for binary classification were selected.

4. Environment Setup

The Medical Image Classification module was developed using **Python 3.11** in a Windows environment. **Visual Studio Code** was used as the primary development environment, while **Jupyter Notebook** was utilized for experimenting with different datasets, training models, and evaluating results. A dedicated **virtual environment** (.venv) was created to isolate project dependencies and ensure reproducibility across different systems.

The implementation relied on **TensorFlow** and **Keras** for deep learning, along with supporting libraries such as **NumPy**, **Pandas**, **Pillow**, **Matplotlib**, and **Scikit-learn** for data preprocessing, visualization, and model evaluation. Since GPU support was not available, all experiments were performed using CPU-based training.

5. Dataset Analysis

Before model training, each dataset was carefully analyzed to verify its structure, annotations, and suitability for binary image classification. The datasets originated from different sources and therefore followed different organizational formats. Some datasets contained annotation files (_classes.csv), while others organized images into class-specific folders. Understanding these differences was necessary to build a reusable training pipeline instead of creating separate scripts for every disease.

During the analysis, several validation checks were performed, including folder structure verification, label verification, image availability, class distribution analysis, and dataset type detection. The datasets were also examined for missing annotations, corrupted files, and class imbalance. Based on these observations, datasets that contained only a single class, lacked valid labels, or had severe imbalance were excluded from training.

Two dataset formats were finally selected for implementation. Glaucoma, Brain Stroke, and Diabetic Retinopathy followed a CSV-based annotation format, whereas the Heart Attack dataset used a folder-based structure. To support both formats efficiently, an automatic dataset detection mechanism was implemented, allowing the same notebook to identify the dataset type and load it using the appropriate preprocessing pipeline.

6. Data Preprocessing

The preprocessing stage was designed to ensure that all datasets produced consistent input regardless of their original format. Every image was resized to **224 × 224 pixels**, converted to RGB format, normalized to improve training stability, and transformed into TensorFlow datasets suitable for batch processing.

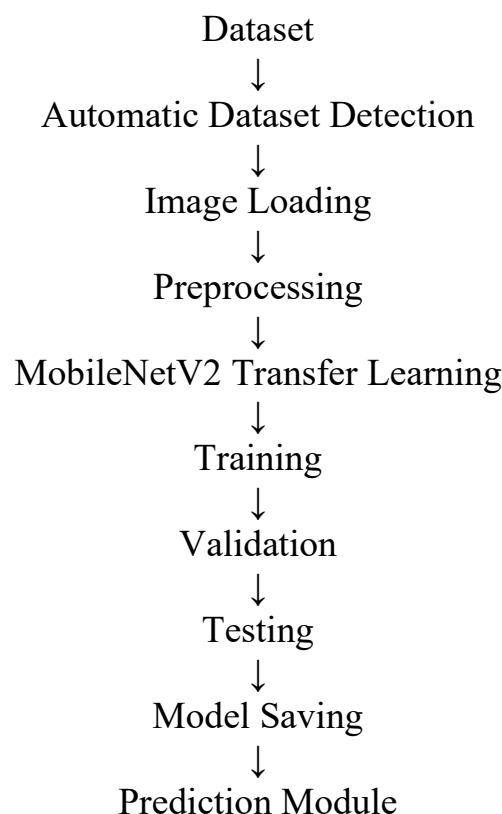
To improve training efficiency, batching, shuffling, caching, and prefetching techniques were incorporated into the data pipeline. Separate preprocessing logic was implemented for CSV-based and folder-based datasets, but both ultimately produced a common dataset format for model training. This approach reduced code duplication and allowed new datasets to be incorporated with minimal modifications.

7. Proposed System

The proposed system provides a reusable workflow for training medical image classification models across multiple diseases. Rather than developing separate training scripts for individual datasets, a unified pipeline was created that automatically detects the dataset format, performs preprocessing, trains the deep learning model, evaluates its performance, and stores the trained model for later use.

The system is based on MobileNetV2 transfer learning, where pretrained ImageNet weights are reused to extract image features. A lightweight classification head consisting of Global Average Pooling, Dropout, and a Dense output layer is added for binary classification. Once training is completed, the best-performing model is automatically saved and can be loaded through the standalone prediction module for inference.

Workflow:



8. Training Pipeline

The training pipeline was implemented using TensorFlow and Keras with **MobileNetV2** as the backbone network. During training, the pretrained convolutional layers were frozen, allowing only the classification layer to be optimized for the selected disease dataset. This approach significantly reduced training time while benefiting from knowledge learned on large-scale image datasets.

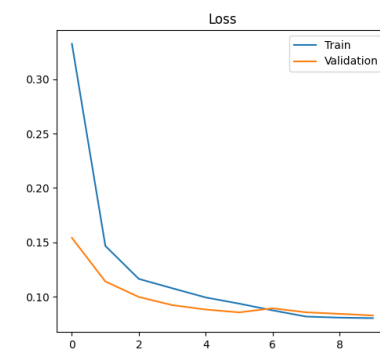
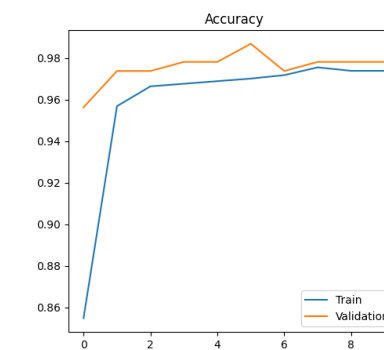
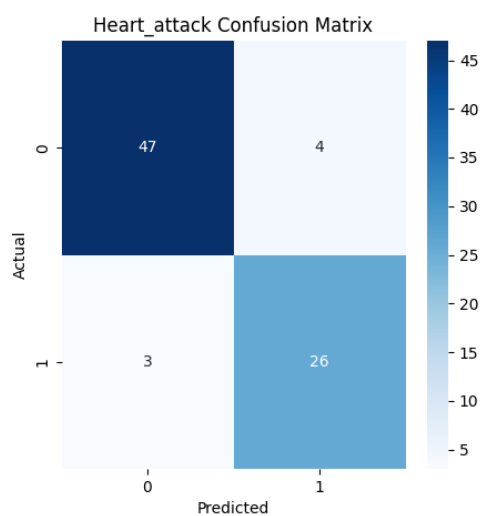
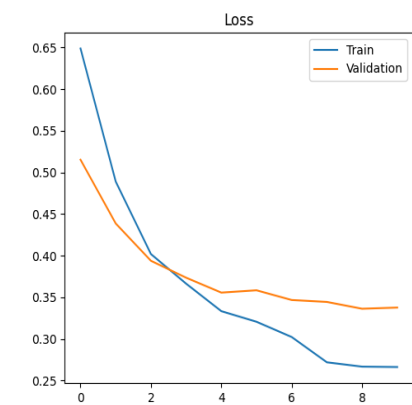
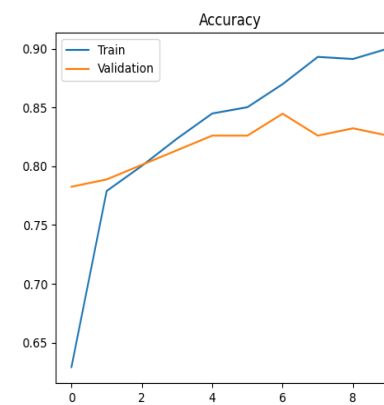
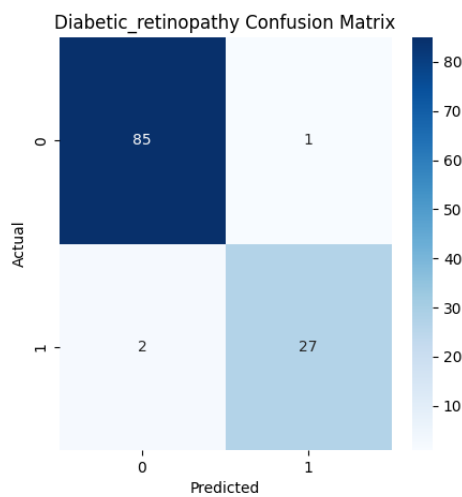
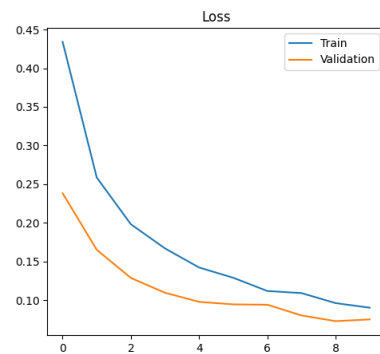
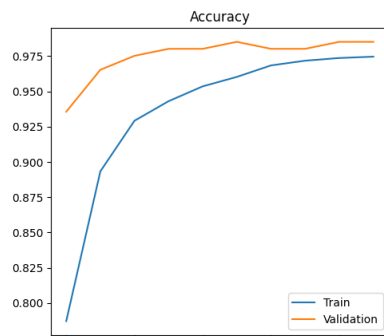
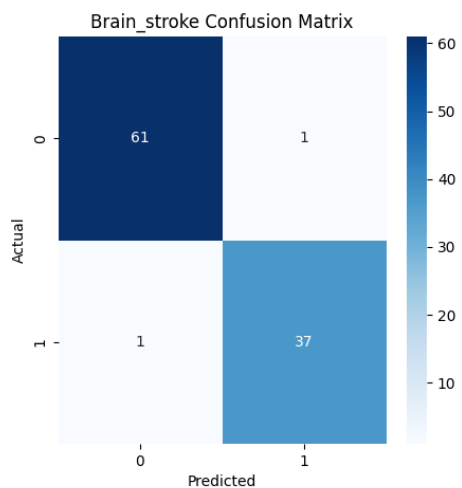
To improve model performance and avoid overfitting, several callbacks were incorporated into the training process. **ModelCheckpoint** automatically stored the best-performing model based on validation accuracy, **EarlyStopping** prevented unnecessary epochs when the model stopped improving, and **ReduceLROnPlateau** dynamically reduced the learning rate whenever validation performance plateaued. After training, the pipeline generated training curves, confusion matrices, classification reports, and saved the trained model in .keras format for deployment.

9. Model Evaluation

The trained models were evaluated using independent test datasets that were not used during training or validation. Performance was measured using multiple evaluation metrics, including accuracy, precision, recall, F1-score, and confusion matrices. These metrics provide a comprehensive understanding of the model's ability to correctly classify both positive and negative cases.

Among the four trained models, Brain Stroke and Diabetic Retinopathy achieved the highest test accuracies of **98.00%** and **97.39%**, respectively. The Heart Attack model achieved **91.25%**, while the Glaucoma model achieved **77.86%**. The comparatively lower accuracy of the Glaucoma model can be attributed to the greater variability and complexity of the dataset. Overall, the trained models demonstrated satisfactory performance and are suitable for integration into the MedAssistAI prediction pipeline.

Disease	Test Accuracy
Glaucoma	77.86%
Brain Stroke	98.00%
Diabetic Retinopathy	97.39%
Heart Attack	91.25%



10. Challenges & Solutions

Challenge 1

Different dataset formats.

Solution

Implemented automatic dataset detection for CSV and folder-based datasets.

Challenge 2

Highly imbalanced datasets.

Solution

Performed dataset analysis and selected suitable datasets for training.

Challenge 3

Incorrect label mapping in folder datasets.

Solution

Verified TensorFlow's alphabetical class assignment and corrected label mapping for inference.

Challenge 4

Reusable inference.

Solution

Developed a standalone `predict.py` module that loads trained models dynamically based on disease type.

12. Conclusion

The image classification module for MedAssistAI was successfully implemented. A reusable training pipeline supporting multiple dataset formats was developed using MobileNetV2 transfer learning. Four disease-specific models were trained and evaluated, and reusable prediction utilities were created for integration with the larger MedAssistAI system. The module is ready for backend integration and further expansion with additional diseases.